

TRABAJO FINAL COMPUTACION DE LA NUEBE

PROFESOR: DANIEL MEJIA

INTEGRANTES:

DANIEL VELASQUEZ, DAVID ORTIZ, MAICOL MONTOYA

TECNOLOGICO DE ANTIOQUIA

MEDELLIN 2026

INTRODUCCION

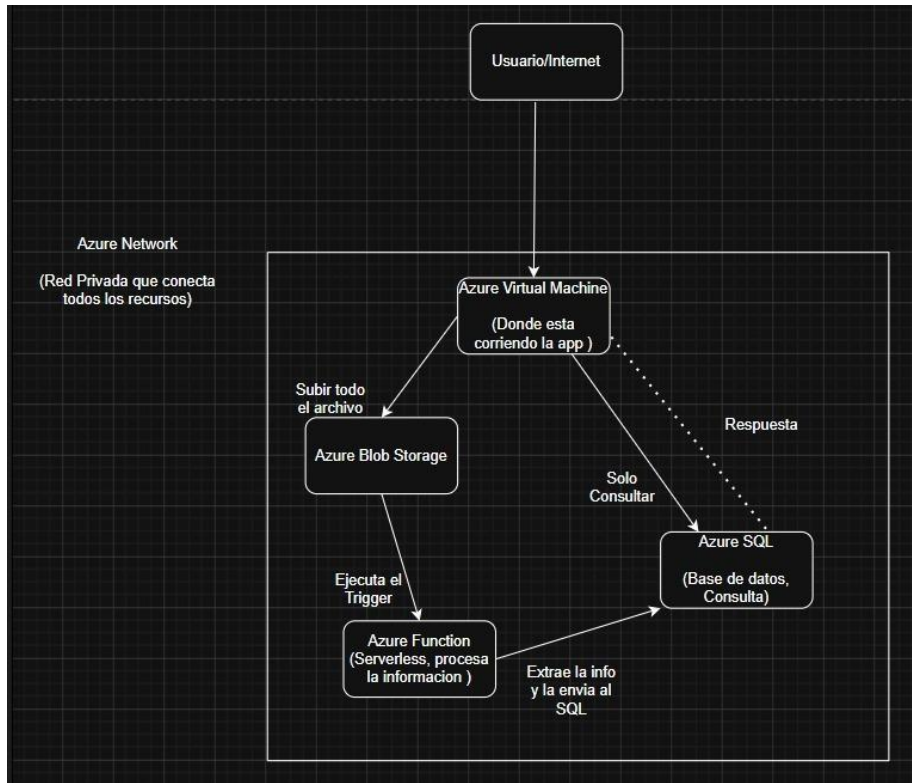
Cuando nos asignaron este proyecto, la primera pregunta que nos hicimos fue ¿cómo podemos construir algo que realmente tenga sentido y no sea solo "levantar servidores por cumplir"? Así fue como llegamos a la idea de crear una aplicación en la nube que resolviera un problema real, subir archivos CSV, procesarlos automáticamente y consultar los datos desde cualquier navegador, sin tocar nada manualmente después de la carga inicial.

Para esto usamos Microsoft Azure como plataforma, no porque fuera la única opción, sino porque nos permitía combinar varios servicios de una forma bastante elegante. La aplicación terminó siendo una mezcla de cinco componentes que trabajan juntos. Una máquina virtual que corre la interfaz web, un Blob Storage donde se guardan los archivos que sube el usuario, una Azure Function que se activa sola cada vez que llega un archivo nuevo y lo procesa, una base de datos SQL donde quedan almacenados los datos, y una red virtual que conecta todo de forma privada y segura.

Lo más interesante del proceso fue ver cómo, al subir un archivo desde el navegador, en cuestión de segundos la Function ya lo había leído, extraído la información y metido los datos en la base de datos, sin que nosotros tuviéramos que hacer nada. Ese momento es básicamente la esencia de lo que significa trabajar con servicios en la nube, automatización real, no simulada.

Este proyecto nos permitió entender de primera mano que la nube no es magia, es arquitectura. Y que cuando los componentes están bien conectados, el resultado habla solo.

Paso 1 Pasos para realizar el proyecto



Lo primero fue definir la arquitectura antes de crear cualquier recurso. En el diagrama se puede ver el flujo completo, el usuario sube un CSV desde la app en la VM, ese archivo va al Blob Storage, la Azure Function lo detecta automáticamente y extrae los datos hacia el SQL, y la VM consulta el SQL para mostrar la respuesta. Todo dentro de una Azure Virtual Network que conecta los recursos de forma privada

Paso 2 Creacion del grupo de recursos

Home

 **PR-ProyectoNube**   ... 

Resource group

Home > Resource Manager | Resource groups

Create a resource group

Basics Tags Review + create

Resource group - A container that holds related resources for an Azure solution. The resource group can include all the resources for the solution, or only those resources that you want to manage as a group. You decide how you want to allocate resources to resource groups based on what makes the most sense for your organization. [Learn more](#)

Subscription * 

Resource group name * 

Region * 

[Previous](#) [Next](#) [Review + create](#)

Primero creamos nuestro grupo de recursos en donde desarrollaremos todo el entregable, le asignamos la Zona y el nombre del grupo de recursos

Paso 3 Creacion del Virtual Network

Home > Network foundation | Virtual networks

Create virtual network

Basics Security Address space Tags Review + create

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * Azure for Students

Resource group * ProyectoNube

[Create new](#)

Instance details

Virtual network name * VNet-Nube

Region * (Canada) Canada Central

[Deploy to an Azure Extended Zone](#)

Home > Network foundation | Virtual networks

Create virtual network

Basics Security Address space Tags Review + create

☐ Allocate using IP address pools. [Learn more](#)

+ Add a subnet

10.0.0.0/16 [Delete address space](#)

10.0.0.0 /16

10.0.0.0 - 10.0.255.255 65,536 addresses

Subnets	IP address range	Size	NAT gateway
default	10.0.0.0 - 10.0.0.255	/24 (256 addresses)	-

[Add IPv4 address space](#)

Advertised address prefixes

Previous Next Review + create

Después de crear el Resource Group llamado ProyectoNube, procedimos a crear la Virtual Network. La nombramos VNet-Nube y la desplegamos en la región Canada Central, que fue la región que usamos para todos los recursos del proyecto.

En la pestaña de Address Space configuramos el rango de IPs de la red en 10.0.0.0/16, lo que nos da 65.536 direcciones disponibles. Dentro de ese rango creamos una subred llamada default con el rango 10.0.0.0/24 para alojar todos los recursos del proyecto.

Paso 4 Crear la Virtual Machine

Home > Compute infrastructure | Virtual machines

Create a virtual machine

Help me create a VM optimized for high availability Help me create a low cost VM Help

Basics Disks Networking Management Monitoring Advanced Tags Review + create

Create a virtual machine that runs Linux or Windows. Select an image from Azure marketplace or use your own customized image. Complete the Basics tab then Review + create to provision a virtual machine with default parameters or review each tab for full customization. [Learn more](#)

i This subscription may not be eligible to deploy VMs of certain sizes in certain regions.

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Azure for Students

Resource group * ⓘ ProyectoNube
[Create new](#)

Instance details

Virtual machine name * ⓘ VM-App ✓

Region * ⓘ (Canada) Canada Central
[Deploy to an Azure Extended Zone](#)

La máquina virtual la nombramos VM-App y la desplegamos en Canada Central, la misma región de todos los recursos, dentro del Resource Group ProyectoNube bajo la suscripción Azure for Students.

Create a virtual machine



Help me create a VM optimized for high availability

Help me create a low cost VM

Help me

Availability zone * ⓘ

Zone 1

✔ You can now select multiple zones. Selecting multiple zones will create one VM per zone. [Learn more](#)

Security type ⓘ

Standard

Image * ⓘ

Ubuntu Server 24.04 LTS - x64 Gen2

[See all images](#) | [Configure VM generation](#)

✔ This image is compatible with additional security features. [Click here to swap to the Trusted launch security type.](#)

VM architecture ⓘ

☐ Arm64

☒ x64

Run with Azure Spot discount ⓘ

☐

Size * ⓘ

Standard_B2ats_v2 - 2 vcpus, 1 GiB memory (US\$ 7,67) (free services eligible)

[See all sizes](#) | [Customize cores](#)

Enable Hibernation ⓘ

☐

ℹ Hibernation is not supported by the size that you have selected. Choose a size that is compatible with Hibernation to enable this feature. [Learn more](#)

el siguiente paso fue crear la máquina virtual que iba a correr la aplicación. Seleccionamos

Ubuntu Server 24.04 LTS como sistema operativo, arquitectura x64, y el tamaño

Standard_B2ats_v2 con 2 vCPUs y 1 GB de memoria, que fue el más económico disponible en

nuestra suscripción. El security type lo dejamos en Standard para no tener restricciones en el

tamaño de la VM.

Microsoft Azure

Search resources, services, and docs (G+/I)

Home > Compute infrastructure | Virtual machines

Create a virtual machine

Help me create a VM optimized for high availabilityHelp me create a low cost VMHelp me

BasicsDisksNetworkingManagementMonitoringAdvancedTagsReview + create

Define network connectivity for your virtual machine by configuring network interface card (NIC) settings. You can control ports, inbound and outbound connectivity with security group rules, or place behind an existing load balancing solution.
[Learn more](#)

Network interface

When creating a virtual machine, a network interface will be created for you.

Virtual network *

(new) VM-App-vnet

Create new

Subnet *

(new) default (10.0.0.0/24)

Public IP *

(new) VM-App-ip

Create new

Public IP addresses have a nominal charge. [Estimate price](#)

NIC network security group

None

Basic

Advanced

Public inbound ports *

None

Allow selected ports

< Previous

Next : Management >

Review + create

En la pestaña de Networking conectamos la VM a la red virtual y le asignamos una IP pública llamada VM-App-ip para que fuera accesible desde internet. En la pestaña de Networking conectamos la VM a la red virtual y le asignamos una IP pública llamada VM-App-ip para que fuera accesible desde internet. El grupo de seguridad de red lo dejamos en Basic y habilitamos los puertos de entrada seleccionados para controlar qué tráfico podía entrar a la máquina

Create a virtual machine



Help me create a VM optimized for high availability

Help me create a low cost VM

✓ Validation passed

Basics

Subscription	Azure for Students
Resource group	ProyectoNube
Virtual machine name	VM-App
Region	Canada Central
Availability options	Availability zone
Zone options	Self-selected zone
Availability zone	1
Security type	Standard
Image	Ubuntu Server 24.04 LTS - Gen2
VM architecture	x64
Size	Standard B2ats v2 (2 vcpus, 1 GiB memory)
Enable Hibernation	No
Authentication type	Password
Username	AzureUser12
Public inbound ports	SSH, HTTP
Azure Spot	No

Disks

Antes de crear la VM Azure hace una validación automática de toda la configuración. En esta pantalla se puede ver el resumen completo con todo lo que configuramos, la validación pasó exitosamente y la VM quedó lista para desplegarse con Ubuntu Server 24.04 LTS, tamaño Standard B2ats v2, autenticación por contraseña con el usuario AzureUser12, y los puertos SSH y HTTP habilitados para acceso desde internet.

Paso 5 Entrar a la Virtual Machine

```
AzureUser12@VM-App:~$ sudo apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2050 kB]
Get:14 http://azure.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [360 kB]
Get:15 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [178 kB]
Get:16 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1694 kB]
Get:17 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe Translation-en [330 kB]
Get:18 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:19 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 c-n-f Metadata [34.5 kB]
Get:20 http://azure.archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3278 kB]
Get:21 http://azure.archive.ubuntu.com/ubuntu noble-updates/restricted Translation-en [760 kB]
Get:22 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [44.7 kB]
Get:23 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse Translation-en [11.3 kB]
Get:24 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:25 http://azure.archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 c-n-f Metadata [656 B]
Get:26 http://azure.archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [40.6 kB]
Get:27 http://azure.archive.ubuntu.com/ubuntu noble-backports/main Translation-en [9172 B]
Get:28 http://azure.archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [5736 B]
Get:29 http://azure.archive.ubuntu.com/ubuntu noble-backports/main amd64 c-n-f Metadata [368 B]
Get:30 http://azure.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [31.0 kB]
Get:31 http://azure.archive.ubuntu.com/ubuntu noble-backports/universe Translation-en [18.6 kB]
Get:32 http://azure.archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [10.5 kB]
Get:33 http://azure.archive.ubuntu.com/ubuntu noble-backports/universe amd64 c-n-f Metadata [1404 B]
Get:34 http://azure.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [212 B]
Get:35 http://azure.archive.ubuntu.com/ubuntu noble-backports/restricted amd64 c-n-f Metadata [116 B]
Get:36 http://azure.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Packages [748 B]
Get:37 http://azure.archive.ubuntu.com/ubuntu noble-backports/multiverse Translation-en [340 B]
Get:38 http://azure.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Get:39 http://azure.archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 c-n-f Metadata [116 B]
```

Una vez creada la VM nos conectamos a ella por SSH y lo primero que hicimos fue actualizar los paquetes del sistema con el comando `sudo apt update`, asegurándonos de tener todo al día antes de instalar las dependencias del Proyecto

```
AzureUser12@VM-App:~$ pip3 install flask azure-storage-blob pyodbc
error: externally-managed-environment

× This environment is externally managed
  ↳ To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
    Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
    sure you have python3-full installed.

    If you wish to install a non-Debian packaged Python application,
    it may be easiest to use pipx install xyz, which will manage a
    virtual environment for you. Make sure you have pipx installed.

    See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You
your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
AzureUser12@VM-App:~$
```

Al intentar instalar las librerías con `pip3` el sistema bloqueó la instalación porque Ubuntu 24.04 protege el entorno de Python por defecto. El error indicaba que el entorno estaba siendo manejado externamente, lo que nos obligó a agregar la flag `--break-system-packages` al comando para forzar la instalación correctamente

```

AzureUser12@VM-App:~$ pip3 install flask azure-storage-blob pyodbc --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting azure-storage-blob
  Downloading azure_storage_blob-12.29.0-py3-none-any.whl.metadata (26 kB)
Collecting pyodbc
  Downloading pyodbc-5.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Requirement already satisfied: click>=8.1.2 in /usr/lib/python3/dist-packages (from flask) (8.1.6)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Requirement already satisfied: Jinja2>=3.1.2 in /usr/lib/python3/dist-packages (from flask) (3.1.2)
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/lib/python3/dist-packages (from flask) (2.1.5)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting azure-core>=1.37.0 (from azure-storage-blob)
  Downloading azure_core-1.41.0-py3-none-any.whl.metadata (49 kB)
  55.8/59.0 kB 3.4 MB/s eta 0:00:00
Requirement already satisfied: cryptography>=2.1.4 in /usr/lib/python3/dist-packages (from azure-storage-blob) (41.0.7)
Requirement already satisfied: typing-extensions>=4.6.0 in /usr/lib/python3/dist-packages (from azure-storage-blob) (4.10.0)
Collecting isodate>=0.6.1 (from azure-storage-blob)
  Downloading isodate-0.7.2-py3-none-any.whl.metadata (11 kB)
Requirement already satisfied: requests>=2.21.0 in /usr/lib/python3/dist-packages (from azure-core>=1.37.0->azure-storage-blob) (2.31.0)
Download flask-3.1.3-py3-none-any.whl (103 kB)
  103.4/103.4 kB 2.1 MB/s eta 0:00:00
Download azure_storage_blob-12.29.0-py3-none-any.whl (434 kB)
  434.8/434.8 kB 20.9 MB/s eta 0:00:00
Download pyodbc-5.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (300 kB)
  300.3/300.3 kB 22.4 MB/s eta 0:00:00
Download azure_core-1.41.0-py3-none-any.whl (220 kB)
  220.9/220.9 kB 10.2 MB/s eta 0:00:00
Download blinker-1.9.0-py3-none-any.whl (9.5 kB)
Download isodate-0.7.2-py3-none-any.whl (22 kB)
Download itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Download werkzeug-3.1.8-py3-none-any.whl (226 kB)
  226.5/226.5 kB 15.6 MB/s eta 0:00:00
Installing collected packages: werkzeug, pyodbc, itsdangerous, isodate, blinker, azure-core, flask, azure-storage-blob
WARNING: The script flask is installed in '/home/AzureUser12/.local/bin' which is not on PATH.

```

Con la flag correcta las librerías se instalaron sin problema. En esta captura se puede ver cómo pip3 descargó e instaló Flask, azure-storage-blob y pyodbc junto con todas sus dependencias, dejando la VM lista para correr la aplicación

Paso 6 Creacion del Blob Storage

Microsoft Azure

Search resources, services, and docs (G+)

Home > Storage center | Blob Storage

Create a storage account

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription *

Azure for Students

Resource group *

ProyectoNube

[Create new](#)

Instance details

Storage account name *

storagenube2026

Region *

(Canada) Canada Central

[Deploy to an Azure Extended Zone](#)

Preferred storage type

Choose preferred storage type

This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance *

☒ **Standard:** Recommended for most scenarios (general-purpose v2 account)

☐ **Premium:** Recommended for scenarios that require low latency.

Redundancy *

Locally redundant storage (LRS)

Previous

Next

Review + create

Con la VM lista, el siguiente componente fue el Azure Blob Storage. Creamos una cuenta de almacenamiento llamada storagenube2026 en Canada Central, con rendimiento Standard y redundancia LRS que es la opción más económica y suficiente para el proyecto. Este es el servicio donde la aplicación deposita los archivos CSV que sube el usuario

Create a storage account ...

Basics

Subscription	Azure for Students
Resource group	ProyectoNube
Location	Canada Central
Storage account name	storagenube2026
Primary service	
Performance	Standard
Replication	Locally redundant storage (LRS)

Advanced

Enable hierarchical namespace	Disabled
Enable SFTP	Disabled
Enable network file system v3	Disabled
Allow cross-tenant replication	Disabled
Access tier	Hot
Managed Identity for SMB	Disabled

Azure Files

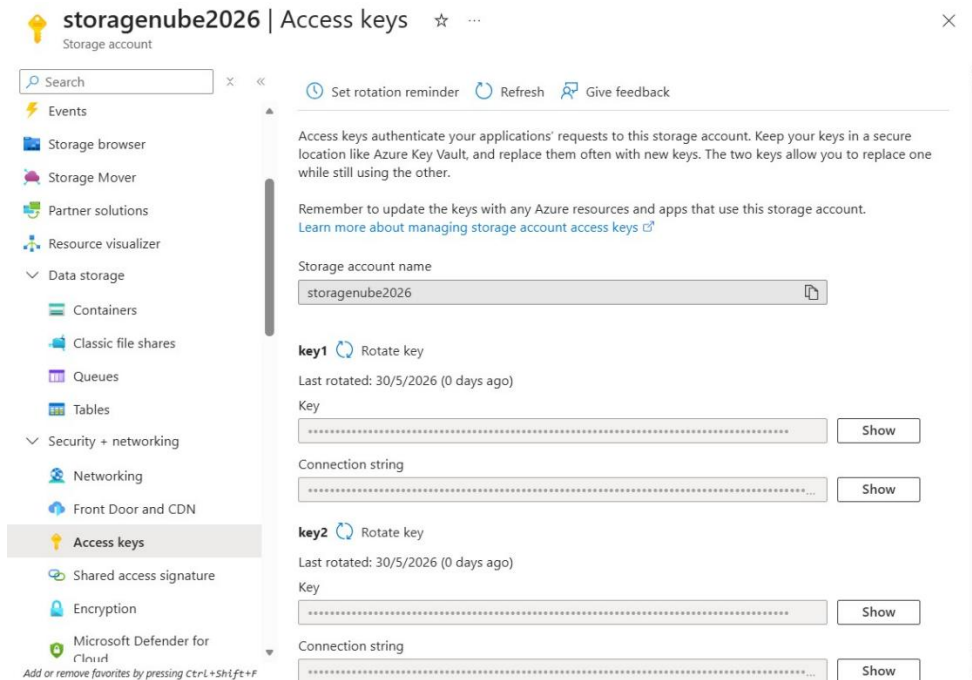
Require Encryption in Transit for SMB	Enabled
---------------------------------------	---------

Previous

Next

Create

Esta es la pantalla de resumen del Storage Account antes de crearlo. Se puede confirmar toda la configuración, nombre storagenube2026, rendimiento Standard, replicación LRS y el Access tier en Hot que es el adecuado para archivos que se acceden con frecuencia. Con todo verificado se procedió a crear el recurso



Una vez creado el Storage Account, obtuvimos las credenciales de acceso desde la sección Access keys. De aquí copiamos la Connection string de key1, que es la que usamos en el código de la aplicación para autenticar las operaciones de subida de archivos al contenedor

Paso 7 Creacion de la Function (Serveless)

Home > Function App > Create Function App

Create Function App (Consumption (Windows))

Subscription *

Resource Group *
[Create new](#)

Instance Details

Function App name *
-fucpgdczhubmdaad.canadacentral-01.azurewebsites.net

☒ Secure unique default hostname on. [More about this update](#)

Operating System
Windows
For Linux, [learn more](#)

Flex Consumption is now the recommended serverless hosting plan for Azure Functions. [Learn more.](#)

Runtime stack *

Version *

Region *

[Review + create](#) [< Previous](#) [Next: Storage >](#)

El siguiente componente fue la Azure Function. La creamos bajo el plan Consumption en Windows, que es completamente gratuito ya que solo cobra cuando la función se ejecuta. La nombramos func-procesar-csv, usamos Node.js 22 LTS como runtime y la desplegamos en Canada Central para mantener todo en la misma region, confirmamos que todo esta OK

Paso 8 Revision de lo creado hasta ahora

Subscription : [Azure for Students](#)

Resource group : [ProyectoNube](#)

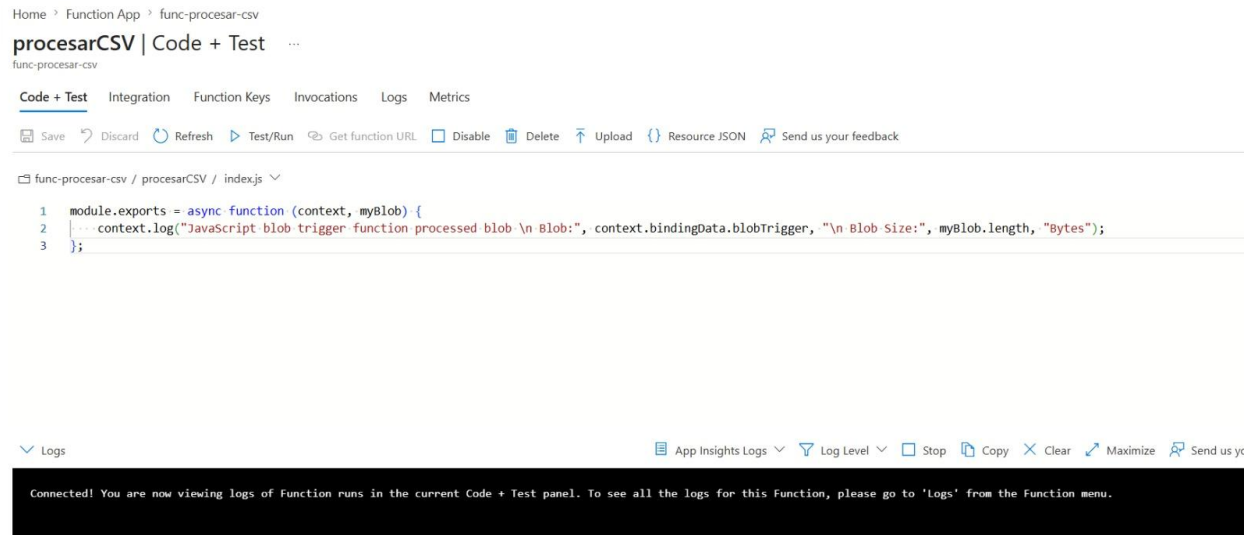
Correlation ID : ecb602ab-a94d-4e23-b5c9-e6fdc62bade0

Deployment details

Resource	Type	Status	Operation details
func-procesar-csv/ftp	Microsoft.Web/sites/basicPublis	OK	Operation details
func-procesar-csv/scm	Microsoft.Web/sites/basicPublis	OK	Operation details
func-procesar-csv	Microsoft.Web/sites	OK	Operation details
func-procesar-csv	Application Insights	OK	Operation details
func-procesar-csv	Application Insights	OK	Operation details
proyectonube9c54	Storage account	OK	Operation details
ASP-ProyectoNube-af50	Microsoft.Web/serverfarms	OK	Operation details
proyectonube9c54	Storage account	OK	Operation details
newWorkspaceTemplate	Deployment	OK	Operation details

> Next steps

Paso 9 Creacion de la Funcion ProcesarCSV



Una vez creada la Function App, creamos la función procesarCSV dentro de ella. En la sección Code + Test escribimos el código en JavaScript que se encarga de leer el archivo CSV cuando llega al Blob Storage, extraer cada fila y enviar los datos a Azure SQL. En la parte inferior se puede ver la consola de logs conectada en tiempo real para monitorear cada ejecución

Paso 10 Creacion del Azure SQL

Microsoft Azure

Search resources, services, and docs (G+)

Home > Azure SQL | SQL databases > Create SQL Database

Create SQL Database Server

Microsoft

Server details

Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.

Server name *

servidor-nube2026

✓

.database.windows.net

Location *

(Canada) Canada Central

▼

Authentication

Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Select your preferred authentication methods for accessing this server. We recommend using only Microsoft Entra authentication [Learn more](#) using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#)

Authentication method

☐ Use Microsoft Entra-only authentication

☐ Use both SQL and Microsoft Entra authentication

☒ Use SQL authentication

El último componente fue Azure SQL. Creamos el servidor lógico llamado servidor-nube2026 en Canada Central y seleccionamos autenticación SQL, que es la más sencilla para conectarse desde el código con usuario y contraseña, sin depender de cuentas de Azure AD.

Microsoft Azure

Search resources, services, and docs (G+)

Home > Azure SQL | SQL databases > Create SQL Database

Create SQL Database Server

Microsoft

Server name *

servidor-nube2026

✓

.database.windows.net

Location *

(Canada) Canada Central

▼

Authentication

Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Select your preferred authentication methods for accessing this server. We recommend using only Microsoft Entra authentication [Learn more](#) using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#)

Authentication method

☐ Use Microsoft Entra-only authentication

☐ Use both SQL and Microsoft Entra authentication

☒ Use SQL authentication

Server admin login *

adminuser

✓

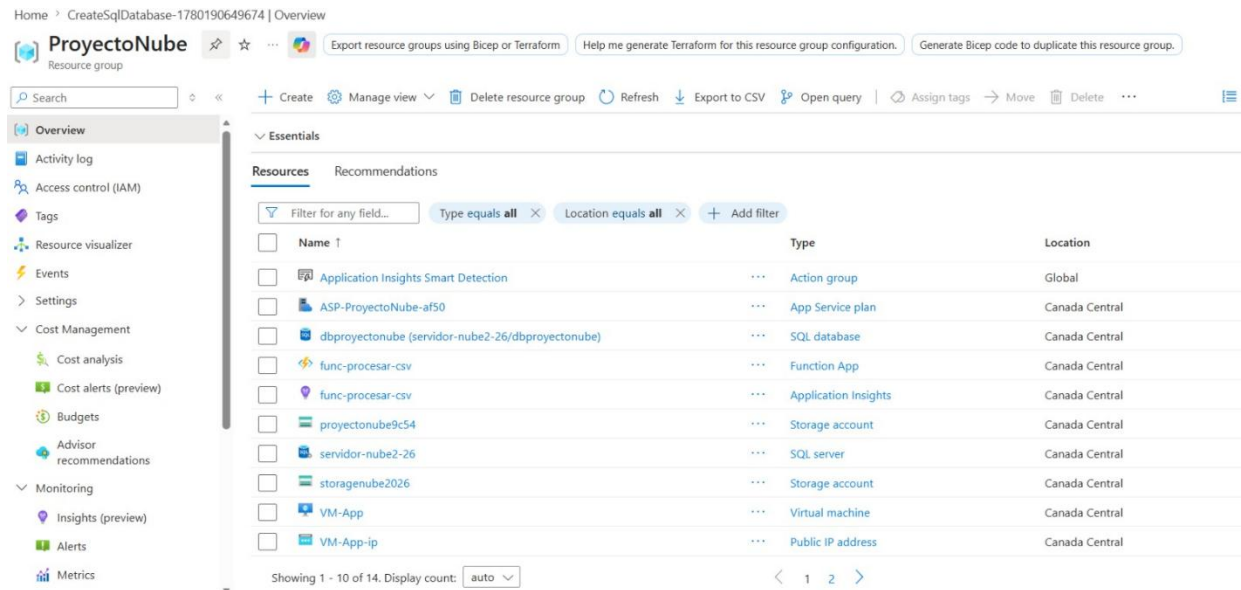
Password *

✓

Confirm password *

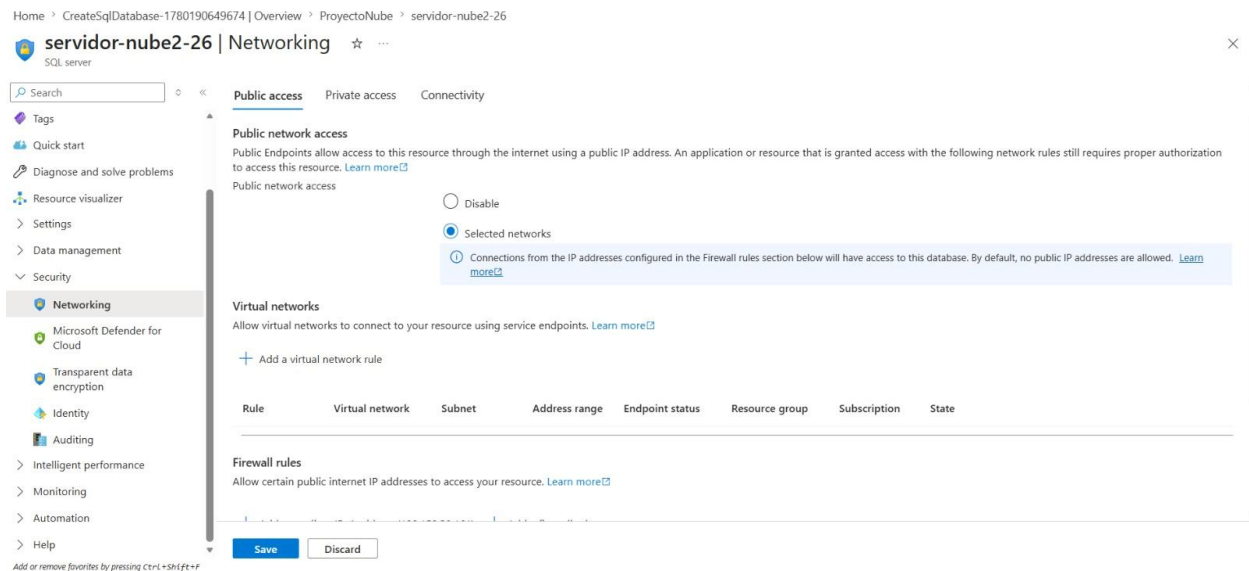
✓

Configuramos las credenciales del servidor SQL con el usuario adminuser y una contraseña segura. Estas son las mismas credenciales que usamos después en el código de la aplicación y de la Azure Function para conectarse a la base de datos



Esta captura muestra el resultado final del proyecto. En el Resource Group ProjectoNube se pueden ver todos los componentes creados y funcionando juntos, la VM-App, el Storage Account storagenube2026, la Function App func-procesar-csv, la base de datos dbprojectonube con su servidor SQL, la IP pública de la VM y la red virtual. Todo en Canada Central, todo conectado dentro del mismo grupo de recursos

Paso 11 Configuraciones de las redes necesarias



Para que la VM y la Azure Function pudieran conectarse al SQL fue necesario configurar el firewall del servidor. En la sección Networking del servidor SQL habilitamos el acceso solo a redes seleccionadas y agregamos las reglas de firewall con las IPs permitidas, garantizando que únicamente los recursos del proyecto tuvieran acceso a la base de datos.

Rule name	Start IPv4 address	End IPv4 address
vm-acces	4.229.233.104	4.229.233.104

Agregamos la regla de firewall vm-acces apuntando a la IP pública de la VM, la 4.229.233.104, tanto en el Start como en el End IPv4 address. Esto le dio acceso exclusivo a la VM para conectarse al servidor SQL y consultar los datos desde la aplicación

[more](#) 

Connecting to dbproyectonube on servidor-nube2-26

Microsoft Entra authentication

SQL authentication

Username *

adminuser

Password *

 **Your IP address isn't allowed to access this server**

Add your IP address to the server's firewall rules or select Allowlist IP 190.158.28.161 below. Changes can take up to 5 minutes to take effect.

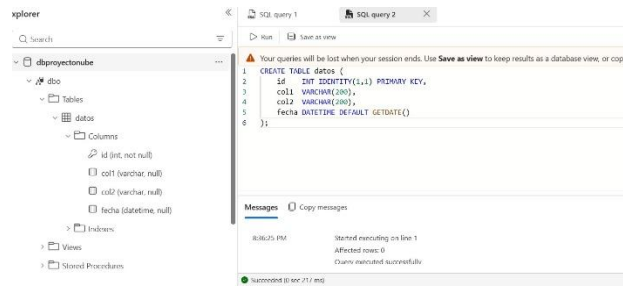
[Allowlist IP 190.158.28.161 on server servidor-nube2-26](#)

Connect

[Forgot password?](#) [Reset in logical server settings](#)

aqui tuvimos un pequeño error en el cual la solución fue darle en el link azul para agregar la ip del computador

Paso 12 Creacion de las tablas



Desde el Query Editor del portal ejecutamos el script para crear la tabla datos con sus cuatro columnas, un id autoincremental como llave primaria, col1 y col2 para guardar el contenido del CSV, y fecha que se registra automáticamente con cada inserción. En el panel izquierdo se puede confirmar que la tabla quedó creada correctamente con todas sus columnas, y en la parte inferior el mensaje Succeeded confirma que la ejecución fue exitosa



despues de ello volvimos a la terminal para ejecutar el commando nano app.py para abrir el editor y pegar el codigo de la app

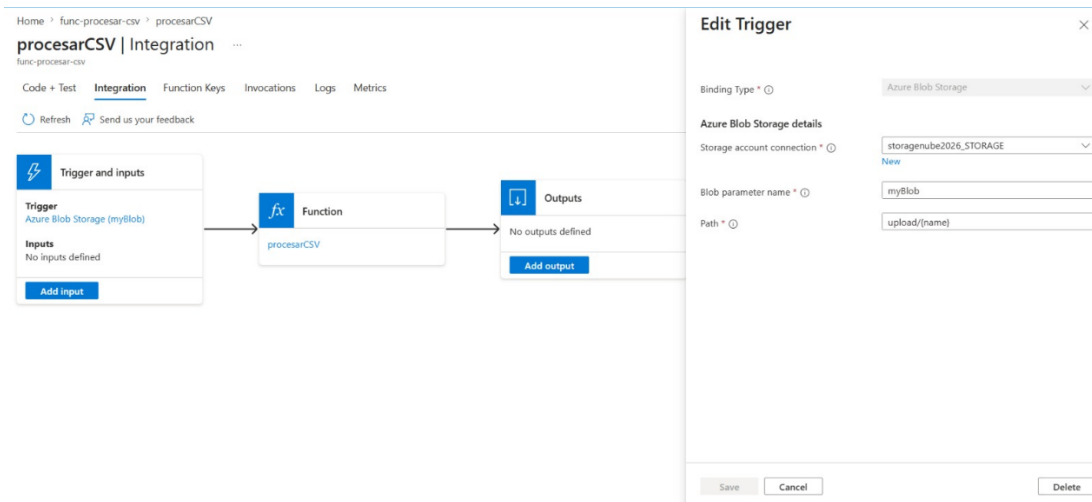
func-procesar-csv / host.json

```
1 {
2   "version": "2.0",
3   "isDefaultHostConfig": true,
4   "extensionBundle": {
5     "id": "Microsoft.Azure.Functions.ExtensionBundle",
6     "version": "[4.*, 5.0.0)"
7   }
8 }
```

despues de ello organizamos nuestras dependencias para que al momento de cargar los cvs se muestren en la parte o seccion de ver datos

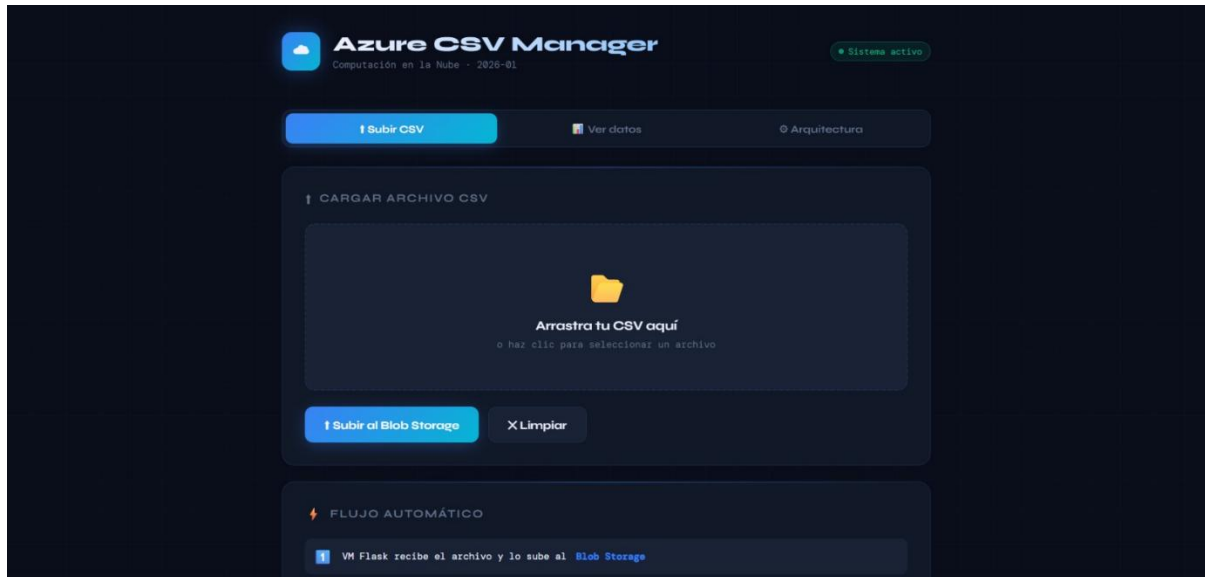
Para que la Function pudiera conectarse al SQL fue necesario instalar la librería mssql. Dado que la consola no permitía ejecutar npm directamente, lo hicimos editando el archivo package.json desde la sección App files de la Function App, agregando mssql como dependencia para que Azure la instalara automáticamente. Para que la Function pudiera conectarse al SQL fue necesario instalar la librería mssql. Dado que la consola no permitía ejecutar npm directamente, lo hicimos editando el archivo package.json desde la sección App files de la Function App, agregando mssql como dependencia para que Azure la instalara automáticamente.

Paso 13 Flujo de trigger



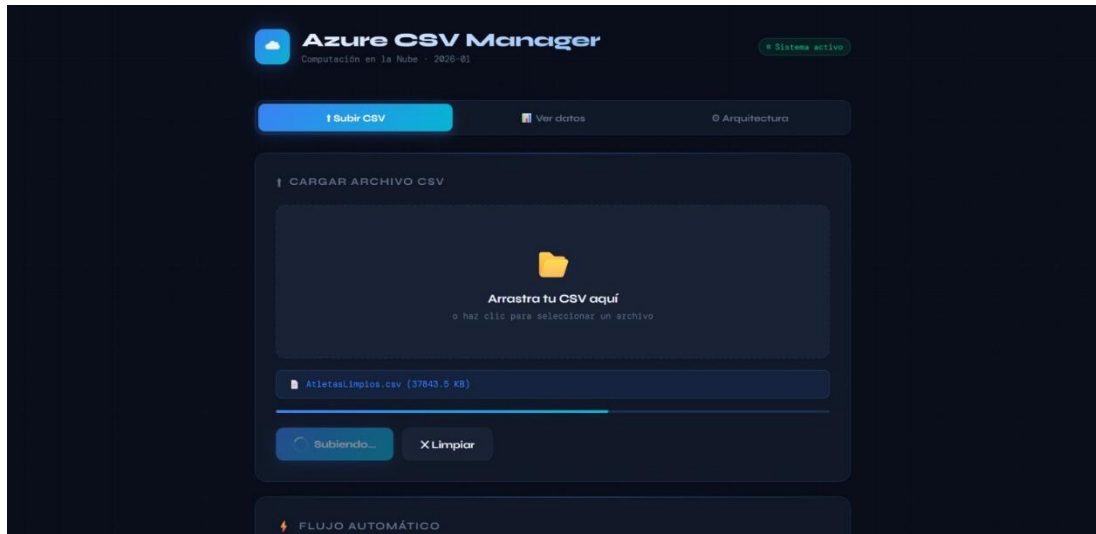
Esta captura muestra la integración completa de la Function. Se puede ver el flujo visual donde el Blob Storage actúa como trigger, dispara la función procesarCSV y esta ejecuta el procesamiento. En el panel de la derecha se confirma que el trigger quedó conectado directamente a storagenube2026 escuchando el contenedor upload, que es donde la VM deposita los archivos CSV

PASO 14 Practica y pruebas

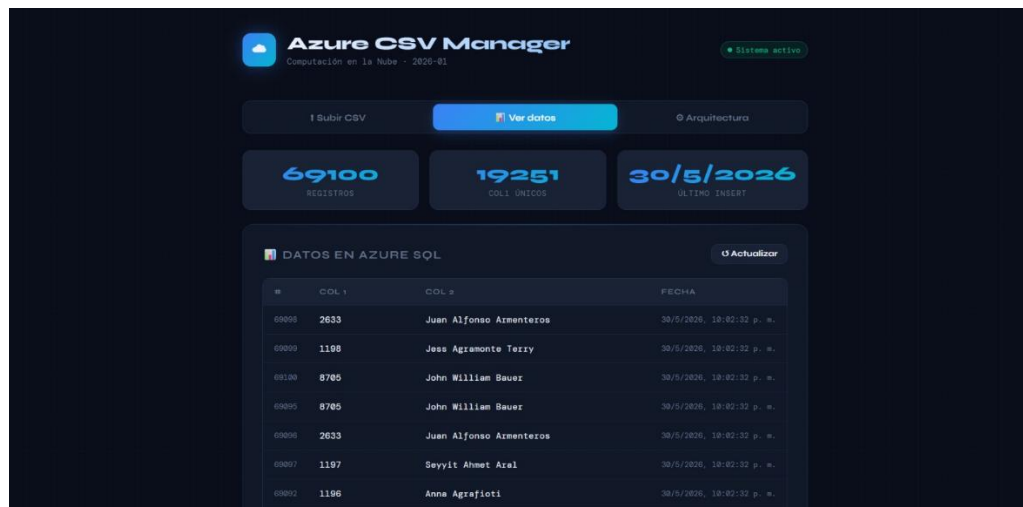


por ultimo volvemos ala terminal y le damos correr al codigo en donde nos llevara con buscar la ip a este web.

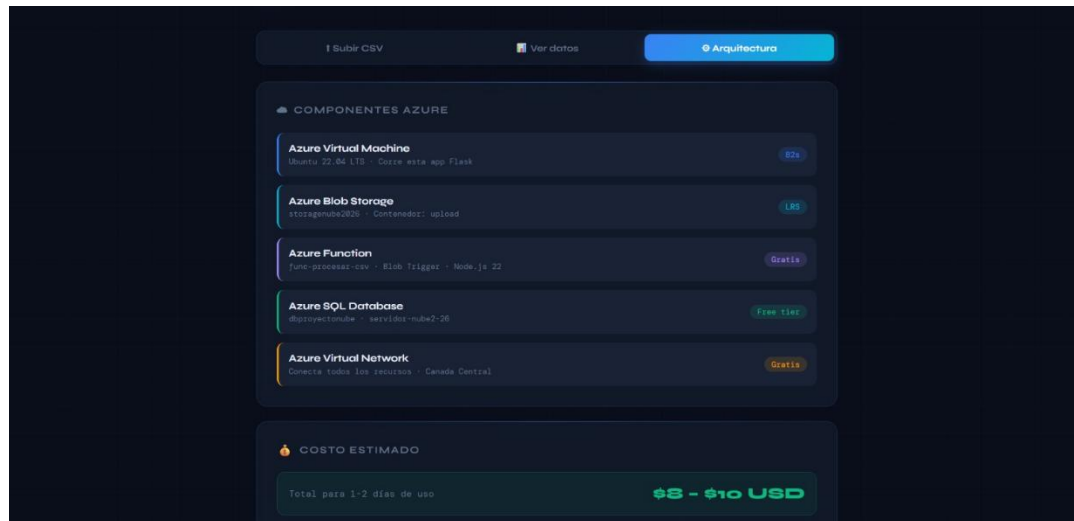
Este es el resultado final de la aplicación corriendo en la VM. La interfaz fue desarrollada con HTML, CSS y JavaScript y muestra tres secciones, la primera para subir el CSV con una zona de arrastre, la segunda para ver los datos ya procesados en el SQL, y la tercera con la información de la arquitectura. En la parte inferior se puede ver el flujo automático que explica al usuario qué ocurre en segundo plano cada vez que sube un archive



Aquí se puede ver la aplicación en pleno funcionamiento. Se seleccionó un archivo real llamado AtletasLimpios.csv de casi 38 MB y la interfaz muestra en tiempo real la barra de progreso de la subida al Blob Storage. El botón cambia automáticamente a "Subiendo..." con un spinner mientras el archivo viaja a la nube, y el botón de Limpiar permite resetear todo si el usuario quiere subir otro archivo. En la parte inferior se mantiene visible el flujo automático para que el usuario entienda qué va a pasar con su archivo una vez termine la carga



Esta captura muestra el resultado final del flujo completo funcionando. En la pestaña Ver datos se pueden ver las estadísticas en tiempo real, 69.100 registros insertados en la base de datos, 19.251 valores únicos en col1, y la fecha del último insert el 30/5/2026. En la tabla se muestran los datos del archivo AtletasLimpios.csv ya procesados y almacenados en Azure SQL, con el nombre del atleta, su código y la fecha exacta en que la Azure Function los insertó automáticamente. Todo esto sin ninguna intervención manual después de haber subido el archive



La pestaña de Arquitectura muestra un resumen visual de todos los componentes usados en el proyecto con su configuración y costo. La VM con Ubuntu corriendo la app Flask, el Blob Storage con redundancia LRS, la Azure Function completamente gratuita, la base de datos SQL en free tier, y la Virtual Network que conecta todo sin costo adicional. Al final se muestra el costo total del proyecto que quedó entre \$8 y \$10 USD para 1 a 2 días de uso, demostrando que es posible construir una solución real en la nube con un presupuesto muy reducido

Conclusiones

Desarrollar este proyecto nos dejó mucho más que una aplicación funcionando en la nube. Nos dejó una forma distinta de pensar sobre cómo se construyen los sistemas modernos.

Lo primero que entendimos es que la nube no es un concepto abstracto, es infraestructura real que se puede tocar, configurar y ver corriendo en tiempo real. Cada vez que subimos un archivo y vimos cómo la Azure Function lo procesaba sola e insertaba 69.100 registros en la base de datos sin que nosotros hiciéramos nada, quedó claro por qué las empresas migran a este modelo.

También aprendimos que construir en la nube implica tomar decisiones de costos desde el principio. No es solo elegir el servicio que funciona, sino el que funciona dentro del presupuesto disponible. Esa restricción nos obligó a investigar, comparar opciones y entender qué cobra Azure y por qué, algo que en un entorno empresarial real es igual de importante que el código.

Los errores que encontramos en el camino, desde la restricción de tamaños de VM en cuentas de estudiante hasta los problemas con el driver ODBC y el firewall del SQL, nos enseñaron más que cualquier guía. Porque en la nube los problemas no son teóricos, son reales y hay que resolverlos.

Por último, este proyecto nos mostró que cinco componentes bien conectados pueden hacer cosas que antes requerían equipos completos de infraestructura. Esa es la promesa de la computación en la nube, y en este proyecto la cumplimos